

ITF23019: Parallel and Distributed Programming

Spring Semester, 2026

Lab 9: Graph Search Algorithms

Submission Deadline: March 18th, 2026 23:59

You need at least 50 pts to pass this lab assignment!

Breadth-first search (BFS) and **Depth-first search (DFS)** are two popular traversal algorithms in graph theory. They are widely used in various artificial intelligence (AI) and operations research applications.

Consider the scenario where you have to find a target node within a graph. Both BFS and DFS provide you with a systematic approach to explore nodes in the graph to find your target. The BFS explores a graph level-by-level while DFS traverses a graph's depth before moving on to its breadth. Note that BFS and DFS only provide you with a sequence of nodes to explore in the graph to find your target. In addition, both DFS and BFS are uninformed search (or blind search), meaning that the search algorithms have no additional information beyond that provided in the problem definition. There is no hint which node is better to visit next. In the worse case, you have to visit all nodes in the graph or the tree. The DFS and BFS algorithms are therefore exhaustive search.

In this lab assignment, you will implement serial version and parallel version of BFS and DFS. For each algorithm, you will find a list of nodes in a given graph that the algorithm explores.

1 Lab Project

The starter code for this lab is provided in the **graph-search** project, which includes several packages. The graph used in this lab is a tree which is already implemented in the **tree** and **utils** packages. Your implementation of DFS and BFS will run on this tree. The **Utils** class generates a tree, and the size of the tree is specified as an input argument. The **Tree** class keeps track of the Tree's **root**. In the **TreeNode** class, each tree node maintains a list of its children, and each child can, in turn, have its own children. Adjacency list is used to make connection between a node and its children. In addition, synchronization mechanism is used to avoid data race problem.

Other packages of **graph-search** project include **bfs_parallel**, **bfs_serial**, **dfs_parallel**, **dfs_serial**, and **main**. Implementation of BFS are in **bfs_parallel**, **bfs_serial** packages while the implementation of DFS are in **dfs_parallel**, **dfs_serial**. You will run the serial and parallel implementations of BFS and DFS in **BfsMain** and **DfsMain** classes within **main** package, respectively. The serial and parallel implementation of BFS and DFS are not yet completed.

For parallel version of these two algorithms, the original tree will be split into multiple sub-trees which roots are the (direct) children of the original root. Then, either serial DFS or serial BFS will be performed on these sub-trees. To implement these algorithms, we use Executor framework with Runnable tasks.

2 Depth-First Search (DFS)

DFS always expands the deepest node in the current frontier of the search tree. The search proceeds immediately to the deepest level of the search tree, where the nodes have no successors. As those nodes are expanded, they are dropped from the frontier, so then the search “backs up” to the next deepest node that still has unexplored successors.

DFS Implementation

- *Serial Implementation:*

For serial implementation of DFS, you can use **Stack** (LIFO queue) or recursive call.

2.1 Use Stack

Figure 1 below explains how the Stack is built and how the visited list is created:

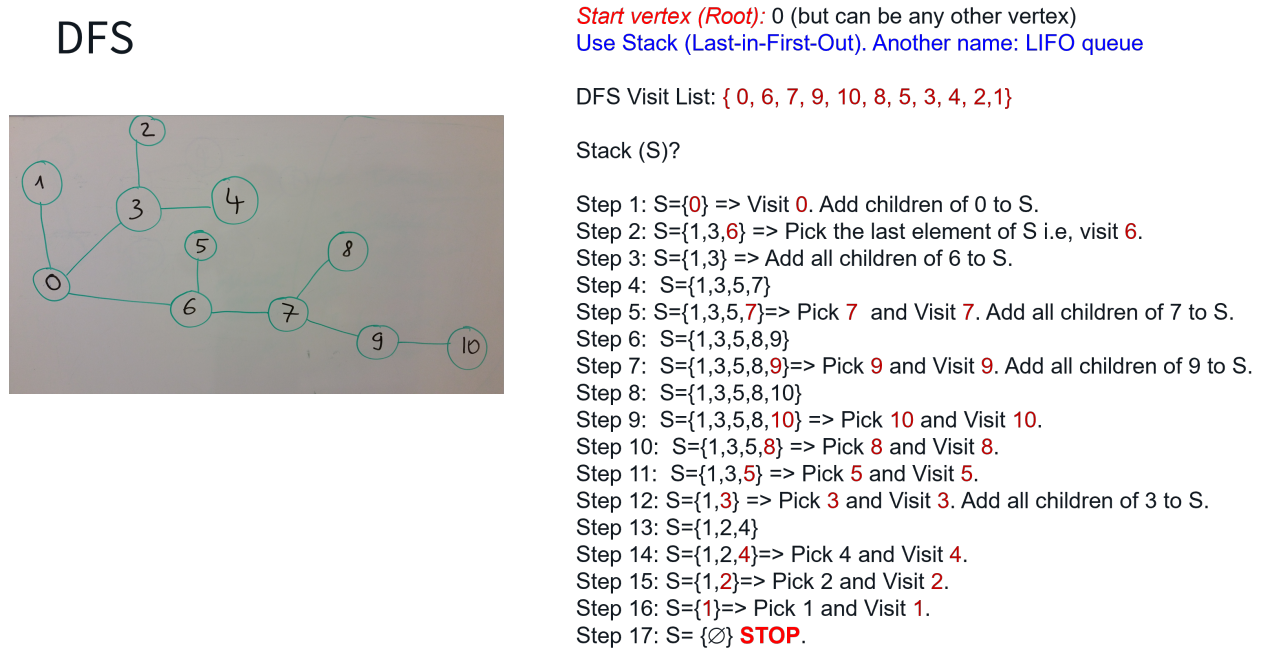


Figure 1: Visting list created using Stack.

- *Serial Implementation:*

You may want to use `add()` and/or `addAll()` methods in Stack Class in Java to implement.

2.2 Use Recursive Call

The pseudo-code for serial implementation of DFS is shown in Figure 2.

- *Parallel Implementation:*

```

DFS-recursive(G, s):
    mark s as visited
    for all neighbours w of s in Graph G:
        if w is not visited:
            DFS-recursive(G, w)

```

Figure 2: Pseudo-code for serial implementation of DFS.

One possible approach to implement parallel DFS is to split the original tree into multiple sub-trees where the root of each sub-tree is a direct child of the original root (see to Figure 3). Each sub-tree will perform serial DFS, and this work is assigned to a **Task** which implements **Runnable** interface. All Tasks will be sent to an **Executor** framework. In other words, we use Executor framework to create a multithread program; each thread (created by implementing Runnable interface) will run serial DFS in its sub-tree. This logic is implemented in the provided code. In the implementation, all children of the root is stored in a concurrent linked queue in **DepthFirstSearchTask** class. Each child in the list will be come the root of a new sub-tree which will be processed currently or in parallel.

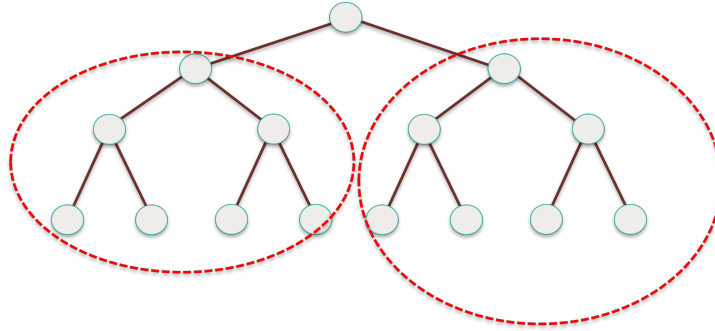


Figure 3: Divide the original Tree into multiple sub-trees each of which is rooted at a direct child of the original Tree.

3 Breadth-First Search (BFS)

The BFS explores a graph level-by-level. It starts at the original node or root of your graph and then moving forward to all neighbors of the current node before proceeding to their neighbors.

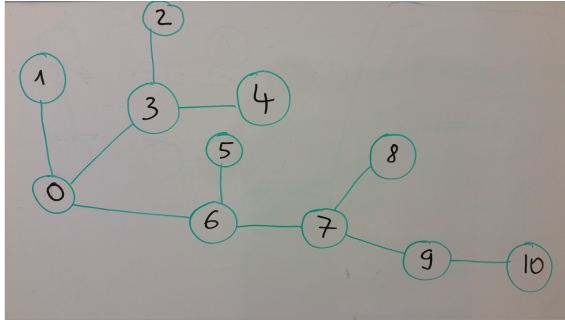
BFS Implementation

- *Serial Implementation:*

To implement serial version of BFS, we will used **Queue** (FIFO) data structure. Refer to **SerialBreadthFirstSearch** class to see how Queue is used and BFS is implemented.

```
Queue<TreeNode> nodes = new LinkedList<>();
```

BFS



Data structure first: adjacent matrix or adjacent list ?

Start vertex: 0 (but can be any other vertex)

Data Structure: Queue (First-in-First-Out)

BFS Visit List = {0,1,3,6,2,4,5,7,8,9,10}

Step 1: Q = {0} => Pick 0 and visit 0. Add 0 to visit list

Step 2: Q = {1,3,6} => Pick 1 and add 1 to visit list

Step 3: Q = {3,6} => Pick 3. Add 3 to visit list and children of 3 to Q.

Step 4: Q = {6,2,4} => Pick 6. Add 6 to visit list and children of 6 to Q.

Step 6: Q = {2,4,5,7} => Pick 2. Add 2 to visit list.

Step 7: Q = {4,5,7} => Pick 4. Add 4 to visit list.

Step 8: Q = {5,7} => Pick 5. Add 5 to visit list.

Step 9: Q = {7} => Pick 7. Add 7 to visit list. Then add children of 7 to Q.

Step 10: Q = {8,9} => Pick 8. Add 8 to visit list.

Step 11: Q = {9} => Pick 9. Add 9 to visit list and children of 9 to Q.

Step 12: Q = {10} => Pick 10. Add 10 to visit list .

Step 13: Q = {∅} **STOP**

Figure 4: Visit list of BFS with serial implementation.

- *Parallel Implementation:*

The idea is almost the same with parallel implementation of DFS. Here, we split the original tree into multiple sub-trees where the root of each sub-tree is a direct child of the original root. Each sub-tree will perform serial BFS, and this work is assigned to a tasks which implements Runnable interface. All tasks will be sent to an **Executor** framework.

4 Exercises

Refer to **graph-search** project provided and analyze the code yourself.

4.1 Exercise 1 (50 pts)

- Complete the serial implementation of BFS in the **SerialBreadthFirstSearch** class. In addition, add command(s) to print out the visiting list of serial BFS. Run the program and include a screenshot in your report.
- Implement parallel BFS. Add your code in the **BreadthFirstSearchTask** class and the **ParallelBreadthFirstSearch** class in the **graph-serach** project. Add command(s) to print out the visiting list of your parallel BFS. Run your program and include a screenshot in your report.
- Add your code in the **BfsMain** class in the **graph-serach** project to compute the speedup of your parallel BFS.
- Often, the speedup of your parallel BFS varies each time you run your code. Therefore, the speedup should be calculated as the average speedup of all runs. Currently, **RUNS** (the number of times you

run your code) is set to 1. If `RUNS` is sufficiently large, you will get a stable speedup.

In `BfsMain` class, change the values of `maxChild` and `numNodes` to resonable values. For which value of `RUNS` do you obtain a stable average speedup, and what is that speedup?

4.2 Exercise 2 (50 pts)

- Implement serial DFS. You can use Stack or recursive call. In addition, add command(s) to print out the visiting list of serial DFS. Run the program and include a screenshot in your report.
- Implement parallel DFS. Add your code in the `DepthFirstSearchTask` class and `ParallelDepthFirstSearch` class in the `graph-serach` project. Add command(s) to print out the visiting list of your parallel DFS. Run your program and include a screenshot in your report.
- Compute speedup of your parallel DFS. Add your code in the `DfsMain` class in the `graph-serach` project.
- Similar to Exercise 2, for which value of `RUNS` do you obtain a stable average speedup, and what is that speedup?
- With the same generated tree, compare the average speedup and memory usage of parallel BFS and parallel DFS in your implementation.

5 What To Submit

Complete the the exercises in this lab assignment. Put your codes inside the `lab9` directory of your repository; we will test your code. Commit your changes and remember to check the GitHub website to make sure all files have been submitted. This lab does not ask for report but you can make a report if you like.